Challenge 05: FileHasher

Category: App Dev Points: 10 Status: SOLVED

Flag

```
MOBISEC{Was_it_known_that_these_one_way_functions_give_you_back_flags?}
```

Source: Verbatim from /apk/81670 analysis log

Description / Methodology

This is an app-development task, not a vulnerability exploit: the target app writes a random file, sends an explicit Intent (action `com.mobisec.intent.action.HASHFILE`) to our exported `HashActivity`, and checks whether the SHA-256 we return matches what it independently computed. The task is to correctly implement that IPC contract.

Correctly implement the receiving side of the IPC contract: hash the file the target points at and return the digest.

How I Got The Flag

1. I decompiled the target and found it sends an explicit Intent (action com.mobisec.intent.action.HASHFILE) to an activity of ours, expecting a SHA-256 hash of a file back.

2. I discovered there was no launcher activity declared, so I added a minimal MainActivity purely so the APK could install and launch.

3. I implemented HashActivity to read the file at the Intent's data URI, hash it with SHA-256 in 8KB chunks, and return the hex digest as a result extra.

4. I built and submitted the APK; the platform's own analysis log confirmed the hash I computed matched exactly what it expected.

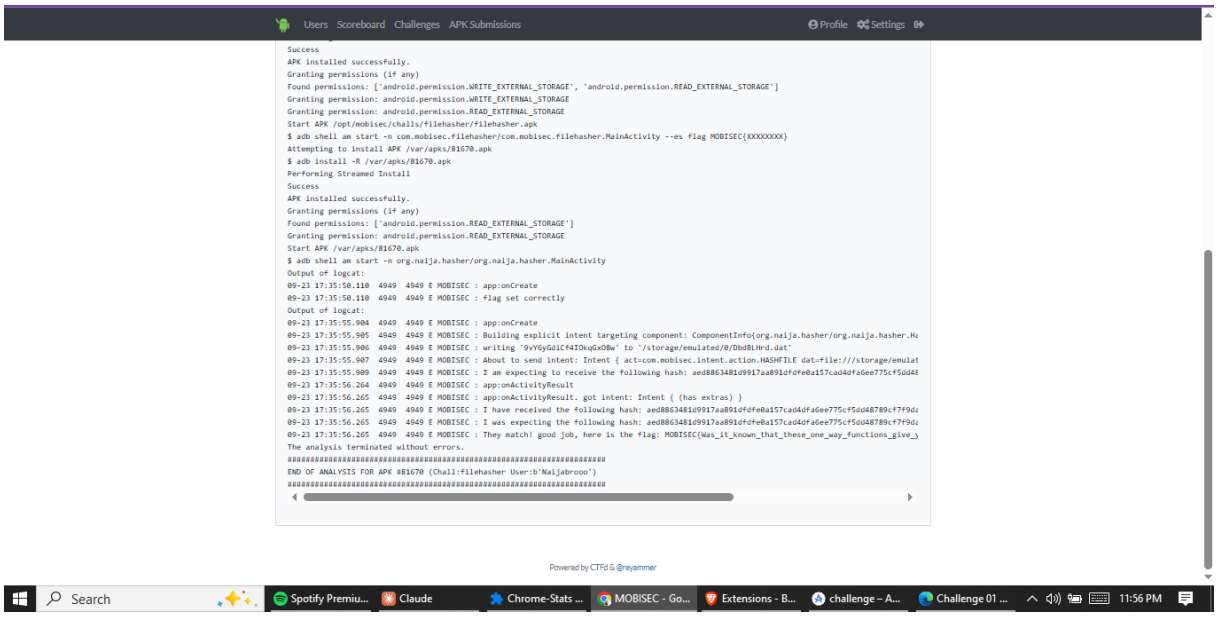
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

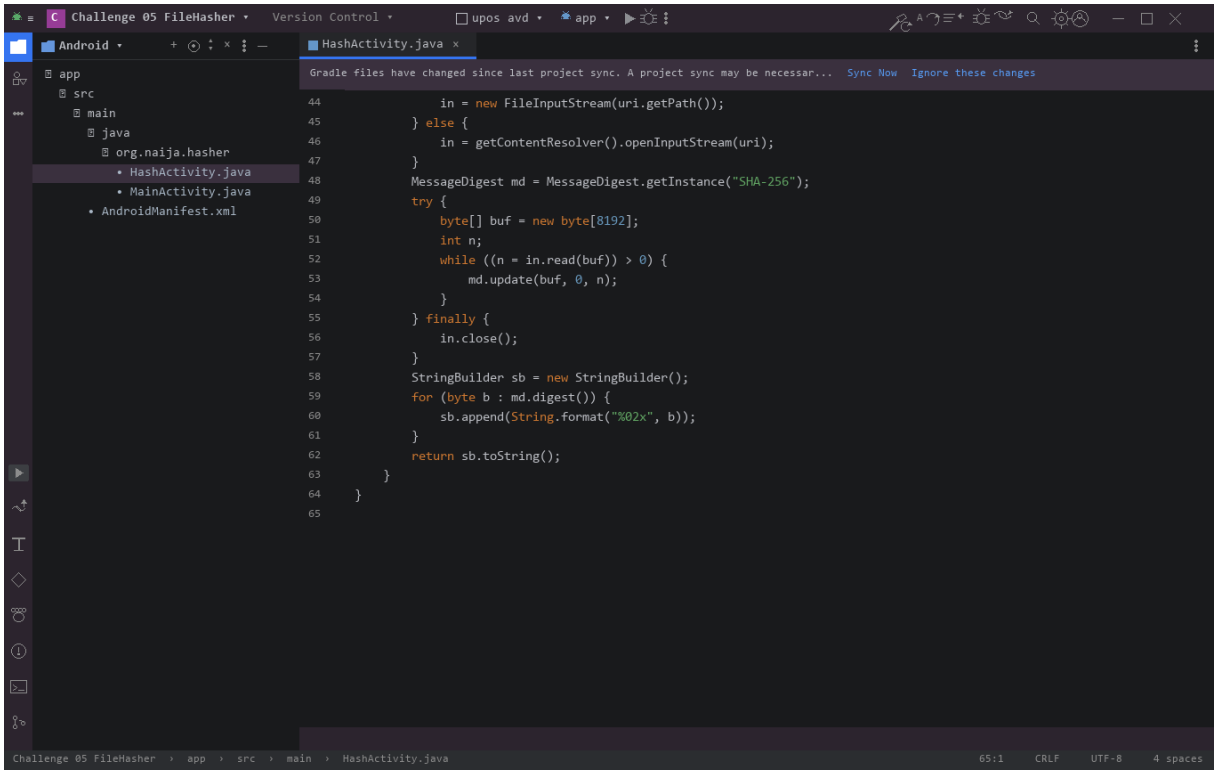
```
$ adb shell am start -n org.naija.hasher/org.naija.hasher.MainActivity
I am expecting to receive the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
I have received the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
I was expecting the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
They match! good job, here is the flag: MOBISEC{Was_it_known_that_these_one_way_functions_give_you_back_flags?}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

HashActivity.java

```
package org.naija.hasher;

import android.app.Activity;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
```

```
import java.io.FileInputStream;
import java.io.InputStream;
import java.security.MessageDigest;
/**
 * Exported activity: takes the URI from the launching Intent's data field,
 * reads that file (either a plain file:// path or via ContentResolver for
 * content:// URIs) and returns its SHA-256 hex digest as the "hash" result
 * extra. Any caller that can start this activity for result gets to hash an
 * arbitrary file it can point a URI at - no permission check is done on the
 * URI itself.
 */
public class HashActivity extends Activity {
    private static final String TAG = "HashActivity";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        Intent result = new Intent();
        try {
            Uri uri = getIntent().getData();
            result.putExtra("hash", calcHash(uri));
            setResult(Activity.RESULT_OK, result);
        } catch (Exception e) {
            Log.e(TAG, "hashing failed", e);
            setResult(Activity.RESULT_CANCELED, result);
        }
        finish();
    }
    // SHA-256 over the file's bytes, streamed in 8KB chunks; returned as lowercase hex.
    private String calcHash(Uri uri) throws Exception {
        InputStream in;
        if ("file".equals(uri.getScheme()) || uri.getScheme() == null) {
            in = new FileInputStream(uri.getPath());
        } else {
            in = getContentResolver().openInputStream(uri);
        }
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        try {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) > 0) {
                md.update(buf, 0, n);
            }
        } finally {
            in.close();
        }
        StringBuilder sb = new StringBuilder();
        for (byte b : md.digest()) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
```